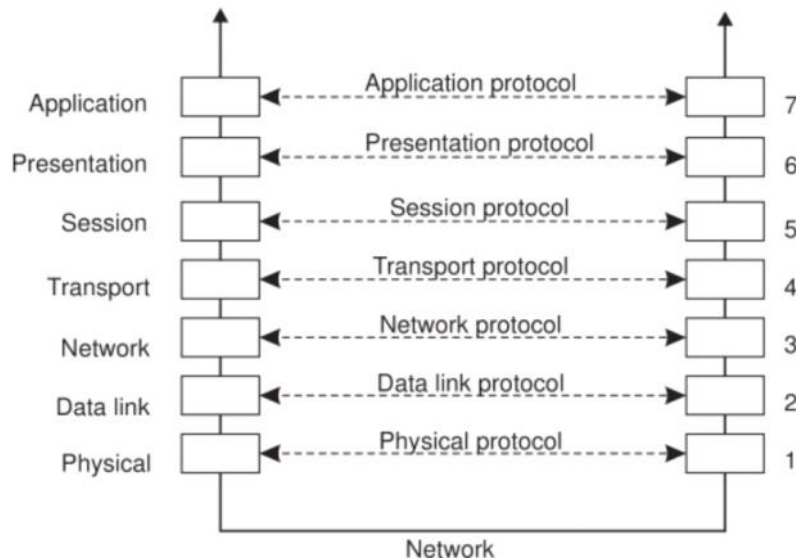


Basic Networking Model



This model shows the traditional 7-layer networking stack (ISO/OSI model) used for communication between systems.

Drawbacks of Basic Networking Model

- **Message-passing focus:** It focuses only on passing messages back and forth.
- **Unwanted functionality:** It often contains functions or features that are not needed for specific applications.
- **Access transparency violation:** It breaks access transparency because users or developers have to know too much about the underlying network details.

Low-Level Layers

Recap of Layers

- **Physical layer:** This layer deals with bits. It defines how bits are built, implemented, and sent between the sender and receiver.
- **Data link layer:** It groups bits into structures called **frames**. It is responsible for handling error control and flow control.
- **Network layer:** It decides and describes how packets should be **routed** across a network of computers.

Observation

- For many distributed systems, the lowest interface they actually interact with is the **network layer**.

Transport Layer

Important Note

- The transport layer is very critical because it provides the **actual communication** facilities that most distributed systems use to function.

Standard Internet Protocols

1. TCP:

- Connection-oriented (establishes a stable link before sending data).
- Reliable (makes sure data arrives safely).
- Stream-oriented communication.

2. UDP:

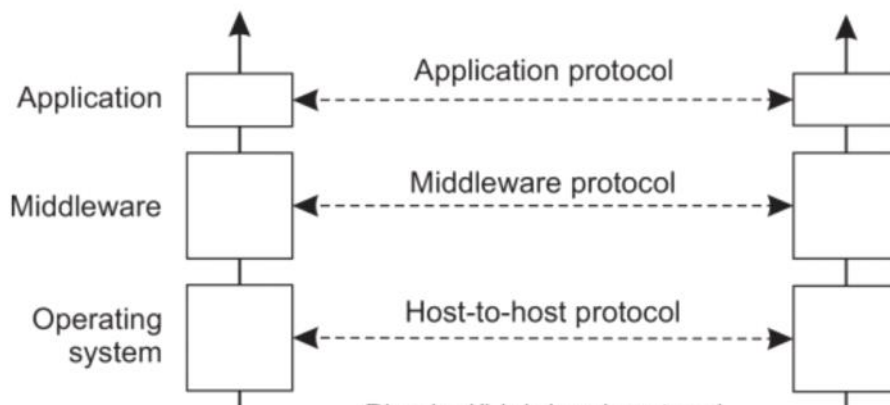
- Unreliable / best-effort communication (does not guarantee data delivery). connectionless.
- Datagram-oriented.

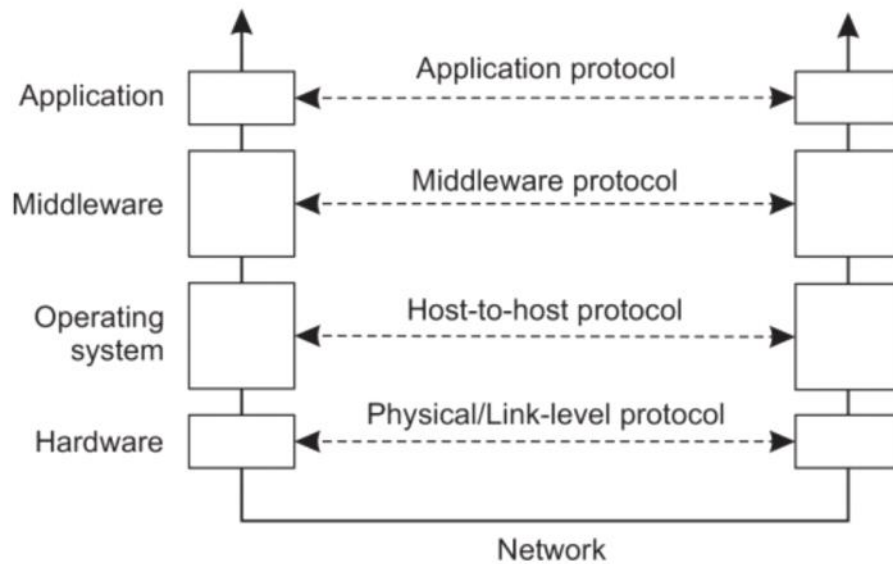
Middleware Layer

- Purpose:** Middleware was invented to provide **common services and protocols**. Many different applications can **reuse** these services instead of building them from scratch.
- Services provided by Middleware:**
 - A rich set of communication protocols.
 - (Un)marshaling of data:** Converting data structures into a format suitable for transmission, which is necessary for integrated systems.
 - Naming protocols:** Allows easy sharing of resources across the system. Applications can find services or objects by name without knowing their exact location.
 - Security protocols:** Ensures secure communication.
 - Scaling mechanisms:** Features like replication and caching

Note

- After middleware handles these common tasks, only **application-specific protocols** are left for the developer to define (e.g., HTTP for web browsers, FTP for file transfers).

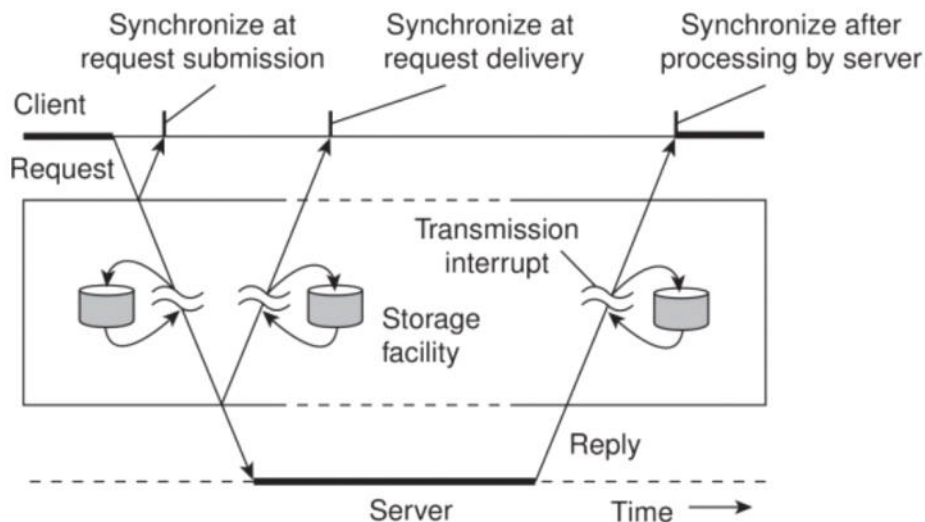




This diagram shows how distributed systems simplify the traditional 7-layer OSI model into 4 practical layers:

- **Application Layer:** Where the specific application software runs.
- **Middleware Layer:** Sits right below the application to give it standard communication and management services.
- **Operating System Layer:** Handles host-to-host protocols and manages the local machine's resources.
- **Hardware Layer:** The physical components and link-level protocols that move the actual data signals.

Types of Communication



When designing a distributed communication system, we must differentiate between two main dimensions:

- **Transient vs Persistent communication:** This is about how the system stores or handles the messages during transmission.
- **Asynchronous vs Synchronous communication:** This is about whether the sender waits or continues its process after sending a message.

Transient vs Persistent Communication

- **Transient communication:** The communication server **discards** the message if it cannot be delivered immediately to the next server or the final receiver. Receiver must be active.
- **Persistent communication:** The communication server **stores (saves)** the message for as long as it takes to deliver it, meaning the receiver does not need to be active when the message is sent.

Places for Synchronization

1. **At request submission:** The client stops waiting and resumes work as soon as the communication system accepts the request.
2. **At request delivery:** The client **waits until** the request successfully reaches the server's side.
3. **After request processing:** The client remains blocked and waits until the server fully finishes the work and sends back a reply.

--

Client/Server Communication

- Traditional Client/Server structures work on a **transient synchronous communication** model.

Key Observations

- **Active Status:** Both the client and the server **must be active** (online) **at the same time** to communicate.
- **Blocking:** The client sends a request and completely stops (blocks) until it gets a reply.
- **Server Role:** The server just waits for incoming requests and processes them one by one.

Drawbacks

- **No multi-tasking:** The client cannot do any other work while waiting for the server's reply.
- **Immediate failure handling:** If something goes wrong, it must be handled instantly because the client is actively blocked and waiting.
- **Not always appropriate:** This model is bad for applications like email (mail) or news systems where instant connection isn't needed.

Messaging (Message-Oriented Middleware)

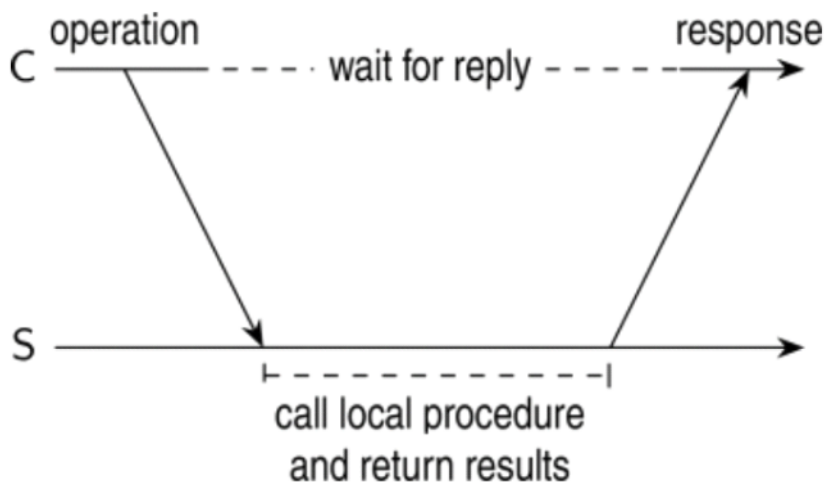
- Message-oriented middleware uses a **high-level persistent asynchronous communication** model.

Key Characteristics

- **Queues:** Processes send messages to each other, and these messages are placed in queues.
- **No waiting:** The sender does not wait for a reply; it can immediately continue doing other tasks.
- **Fault Tolerance:** The middleware layers usually take care of keeping messages safe even if a system crashes.

Basic RPC (Remote Procedure Call) Operation

- Developers already know how to use local functions/procedures (simple procedure model).
- Good procedures work like a **black box** (in isolation).
- Therefore, we can run these procedures on a completely separate machine without changing the main programming logic.

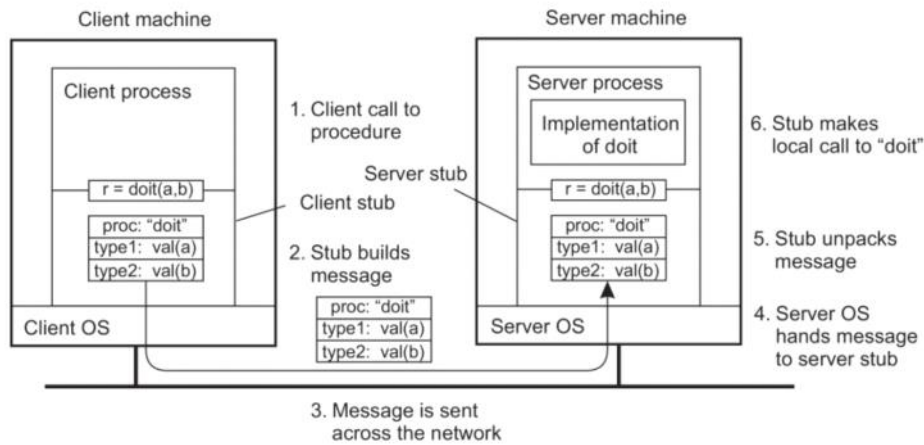


Conclusion: The complex network communication details between the caller (client) and callee (server) are completely **hidden** from the developer by using the standard procedure-call mechanism.

Basic RPC Operation

- **Main Goal:** To run a procedure on a remote machine, but make it look like a local call. It completely **hides** network communication details from the developer.

How it works



:

1. **Client process** calls the **Client stub** (looks like a local call).
2. **Client stub** packs the parameters(marsahlling) into a message and calls the **Client OS**.
3. **Client OS** sends the message over the network to the remote machine.
4. **Server OS** receives the message and gives it to the **Server stub**.
5. **Server stub** unpacks the parameters(unmarshalling) and calls the actual **Server process**.
6. The server runs the code and returns the result back using the same path in reverse.

RPC: Parameter Passing & Limitations

- **The Problem:** Client and server machines can have **different data representations** (like byte ordering).
- **Marshalling:** Transforming a value into a sequence of bytes. Both sides must agree on the same encoding to interpret messages correctly.

Assumptions:

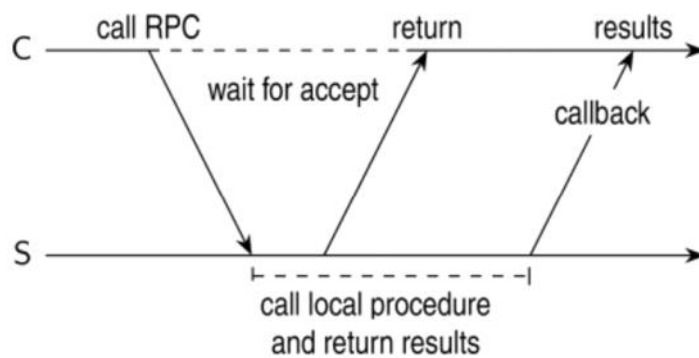
- **Copy in/copy out semantics:** Values are copied; you cannot change parameters directly on the other side during execution.
- All data must be passed as parameters. You cannot pass direct references (pointers) to **global data**.
- **Conclusion:** Because of these differences, **full access transparency cannot be realized** perfectly.
- **Solution:** A **remote reference mechanism** helps. It allows remote pointers to be passed as parameters in RPCs, and **stubs** can sometimes act as these pointers.

Asynchronous RPCs: This approach removes the strict request-reply pattern.

- The client can continue executing its own tasks immediately without waiting for an answer from the server.

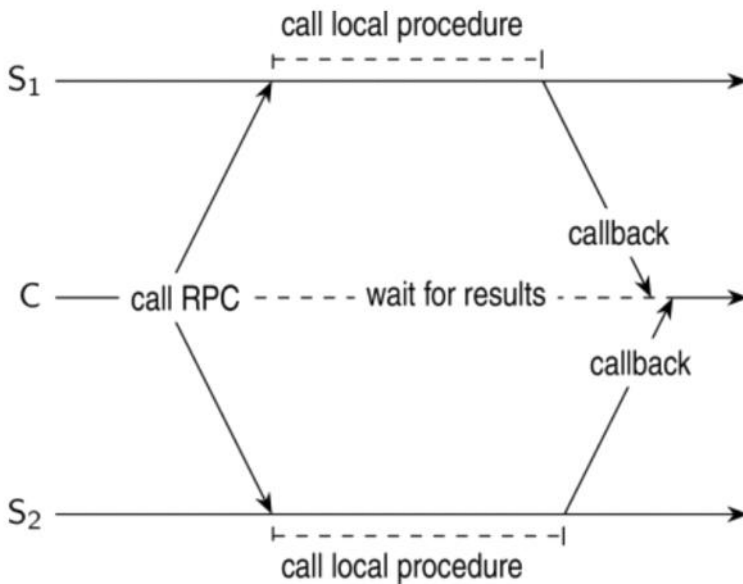
The Process:

- The client makes an RPC call and only waits briefly for an acknowledgment (wait for accept).
- Once accepted, the client gets a return signal and goes back to its work while the server processes the data.
- The final results are delivered later using a callback mechanism.



Sending Out Multiple RPCs : This expands the asynchronous idea to handle multiple destinations by sending an RPC request to a group of servers simultaneously.

- The client (C) triggers the calls to multiple servers (like S_1 and S_2) and continues its timeline while they execute locally.
- **Collection:** The client eventually collects the individual answers asynchronously via independent callback events.

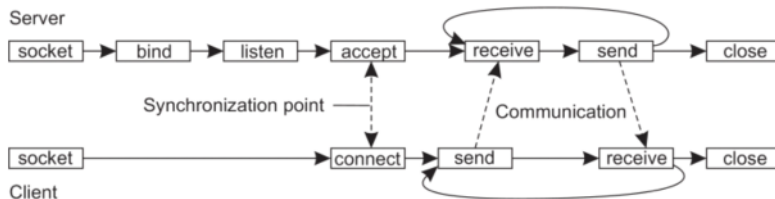


Transient Messaging: Sockets (Berkeley Socket Interface)

The standard Berkeley socket interface relies on a sequence of precise primitive operations to establish connections and move data:

Operation	Description
socket	Create a new communication end point
bind	Attach a local address to a socket
listen	Tell operating system what the maximum number of pending connection requests should be
accept	Block caller until a connection request arrives
connect	Actively attempt to establish a connection
send	Send some data over the connection
receive	Receive some data over the connection
close	Release the connection

Synchronization Point: A connection is only established when the client executes **connect** and the server handles **accept**.



Making Sockets Easier to Work With (ZeroMQ)

- **The Problem (Sockets):** They are too low-level. You must manage everything (connections, errors) manually, which causes mistakes.
- **The Solution (ZeroMQ):** It gives you ready-made communication structures to fix this problem.
- **How it Works:** It pairs sockets together (one to send, one to receive). All communication is **asynchronous**.
- **3 Main Patterns:**

1. Request-reply
2. Publish-subscribe
3. Pipeline

MPI (Message Passing Interface)

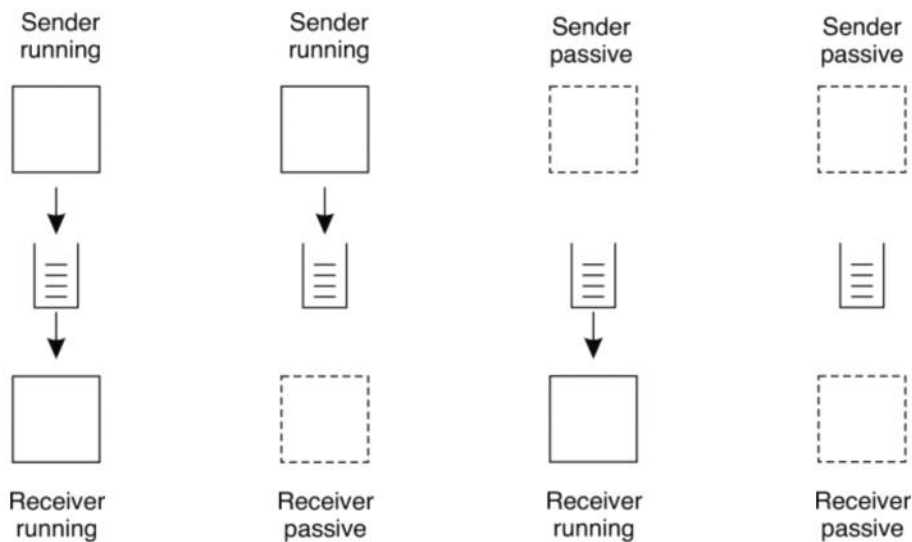
- A standard for high-performance communication in parallel computing and distributed systems.
- It gives developers fine control over when to send, when to wait, and how to handle messages.

Key Operations:

- MPI_BSEND: Add message to a local send buffer.
- MPI_SEND: Send and wait until it is copied to a buffer.
- MPI_SSEND: Send and wait until transmission starts.
- MPI_SENDRECV: Send a message and wait for a reply.
- MPI_ISEND: Send reference and continue (do not wait).
- MPI_ISSEND: Send reference and wait until receipt starts.
- MPI_RECV: Receive a message (blocks if there is no message).
- MPI_IRECV: Check for a message (never blocks).

Queue-Based Messaging

- Senders and receivers use a queue to communicate.



4 Possible Combinations/asynchronous persistent communication

Message-Oriented Middleware (MOM)

- It provides asynchronous persistent communication using middleware-level queues (buffers at communication servers).

Basic Operations:

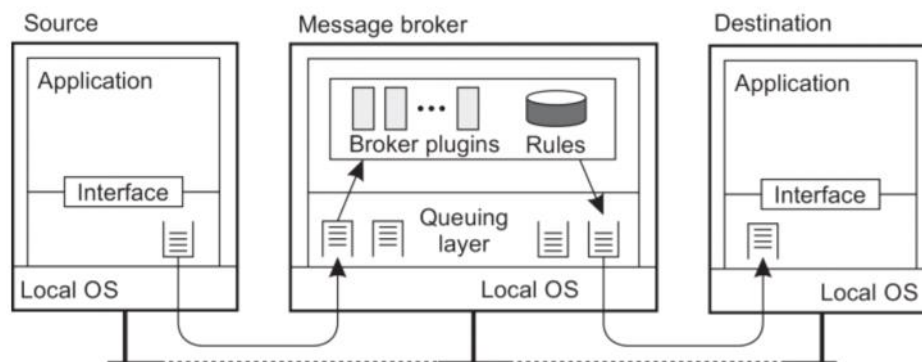
Operation	Description
PUT	Append a message to a specified queue
GET	Block until the specified queue is nonempty, and remove the first message
POLL	Check a specified queue for messages, and remove the first. Never block
NOTIFY	Install a handler to be called when a message is put into the specified queue

General Model: Queue Managers & Routing

- **Local Restriction:** Applications can only put messages into or extract messages from a local queue.
- **Routing Necessity:** Because applications only interact with local queues, queue managers must route messages to move them across the network.
- **Mechanism:** The source queue manager uses an address lookup database to

Message Broker

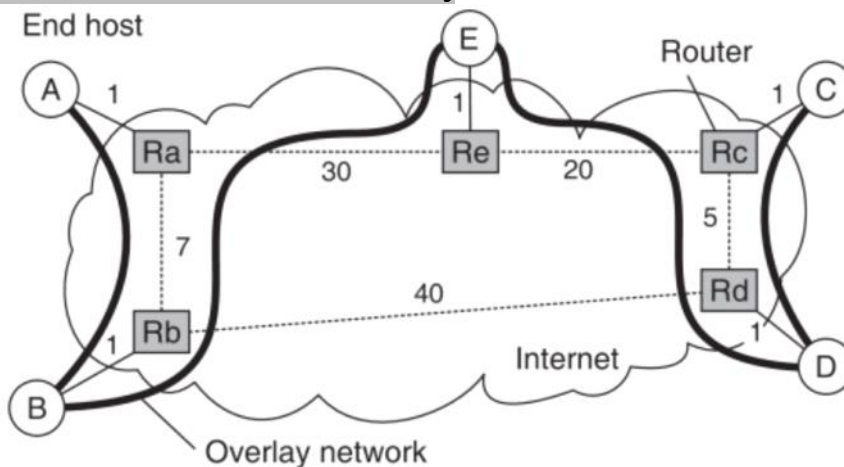
- **The Assumption:** Standard message queuing systems assume all applications agree on a single, common messaging format and protocol.
- **The Solution:** A message broker handles application differences (heterogeneity) by transforming incoming messages into the target format.
- **Role:** It frequently acts as an application gateway.
- **Extra Capability:** It can provide subject-based routing capabilities, such as publish-subscribe features.



Application-Level Multicasting (ALM)

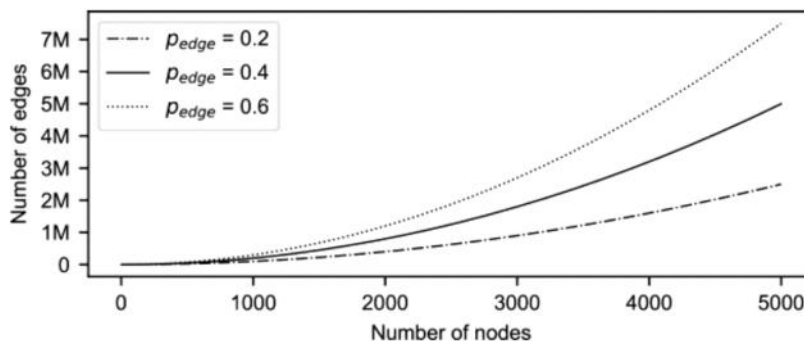
- **Essence:** It organizes nodes of a distributed system into an overlay network to distribute and spread data.
- **Network Structures:**
 - It often uses a **tree** structure, which provides unique paths.
 - Alternatively, it can use **mesh networks**, which require a form of routing.
- **Flexibility:** It gives developers the flexibility to explicitly design how nodes connect and how data is routed using these tree or mesh structures.

ALM: Some Costs - dezavantaj



- **Link Stress:** This measures how many times an ALM message goes through the exact same physical link. For example, a message sent from node A to node D might have to cross the link (Ra, Rb) twice.
- **Stretch:** This is the delay ratio between the ALM-level path and the actual network-level path. For example, if a message from B to C takes a path of length 73 in ALM but only 47 at the network level, the stretch is $73/47$.

Flooding & Variations



- **The Essence:** A node P sends a message m to all of its neighbors. Each neighbor then forwards that message to all of its own neighbors, except back to P, only if it has never seen message m before.
- **The Variation:** To reduce overhead, a node Q can forward a message with a specific probability (p_{flood}). This probability can change based on how many neighbors Q has (its node degree) or the degree of its neighbors.

Epidemic Protocols

- These are probabilistic methods used to share and spread updates across replicas in a distributed system.
- **Basic Assumptions:** It assumes there are no write-write conflicts. Updates happen at a single server, replicas pass updates to only a few neighbors lazily (**not immediately**), and **eventually**, every replica

gets the update.

Two Forms of Epidemics:

- **Anti-entropy:** Replicas regularly choose another random replica to **exchange** differences so they become identical.
- **Rumor spreading:** A newly updated (contaminated) replica actively **tells a few other replicas** about the update to contaminate them too.

Anti-Entropy Mechanism

- **Core Operations:** Node P chooses a random node Q and chooses one of three methods to sync:
 1. **Pull:** P only takes new updates from Q.
 2. **Push:** P only sends its own updates to Q.
 3. **Push-pull:** P and Q exchange updates with each other.
- **Efficiency:** Using the push-pull strategy, it takes $O(\log(N))$ rounds to spread updates to all N nodes in the system.

Deleting Values & Death Certificates

- **The Problem:** You cannot simply erase a value from one server and expect the deletion to spread. Because of how epidemic algorithms work, other servers will eventually bring the deleted value back.
- **The Solution:** Instead of a simple deletion, you must register the removal as a special update by creating a **death certificate**.
- **Removing Death Certificates:** Certificates cannot stay forever, so they are cleaned up using two strategies:
 - Use a global algorithm (like garbage collection) to check if every single server knows about the deletion before destroying the certificate.
 - Set a maximum lifetime for the certificate, assuming it spreads in finite time (though this risks missing some servers).
- **Crucial Note:** It is absolutely necessary that a removal command successfully reaches all servers.